Express Yourself

- Use Word or OpenOffice to create a new document
- Save the file as LastFirst_ic21.doc
  - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 21"
- **Exercise 0**: Open Eclipse
  - Switch your workspace to workspaceWeek14-15
  - Open the **Dot** program
  - First, we'll make our **Dot** class "mouse aware"
  - Then, we'll hook up the buttons in our blackjack game

Java's Event-handling

- Java uses interfaces for all of its event handling
  - These interfaces are called "listeners"
  - Defined in the **java.awt.event** package
- Events are "external" occurrences that tell you something has "happened" to your program:
  - The user has clicked or moved the mouse
  - User has pressed or released a key on the keyboard
  - The user has entered some text into a text field
  - The user has made a selection from a menu
- **Implement interfaces** to respond to these events

How Event Handling Works

- Any widget can generate (or fire) an event
  - Different components fire different kinds of events
- Any class (not just components) can register to receive notification and respond to an event
  - Objects are called "listeners", "subscribers" or "targets"
  - Components maintain a list of all listener objects that should be notified when an event occurs
- To respond to an event, two things must occur:
  - The **class** must implement the correct interface
  - The **object** must register or subscribe to the listener list maintained by a particular object

Step 1: Preparation

- To make our **Dot** class "mouse aware" we must:
  - Import that package that contains the event interfaces
  - Add **implements MouseMotionListener** to the class header

```
13 import java.awt.*;
14 import java.awt.event.*;
15
16 public class Dot extends Component
17         implements MouseMotionListener
18 {
19     private int radius;
20
```

  - As soon as we do that, our code stops compiling

Step 2: Subscribe

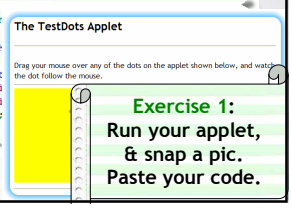
- To **subscribe**, tell the generator to put the responder on the "subscription list" using a specific method
  - **addMouseMotionListener()** for move/drag events
  - Generator is the receiver, subscriber is the argument
  - For the **Dot** class, put this code in the constructor

```
21:     /**
22:      *  Set the default radius, and colors for each Dot
23:      */
24:     public Dot()
25:     {
26:         setRadius(10);
27:         setBackground(Color.green);
28:         setForeground(Color.red);
29:         addMouseMotionListener(this);
30:     }
```

Adding Behavior

- **Nothing happens**! We still need to write code
  - Locate the position of the mouse pointer
  - Convert coordinates into program coordinates
  - Set the location of the dot to the new coordinates

```
51:     /**
52:      * Move the Dot's positio
53:      */
54:     public void mouseDragged(M
55:     {
56:         Point mouseLoc = e.get
57:         mouseLoc.translate(thi
58:                            thi
59:         setLocation(mouseLoc);
60:     }
61:
```



Exercise 1:
Run your applet, & snap a pic.
Paste your code.

Action Events

- Action Events are "fired" when
  - A button is pressed
  - The user presses the ENTER key in a text field
  - A menu selection is made
- Can respond same way we did mouse events, or…
  - Create a "handler" class to deal with the event
  - Tell the widget that fires the event to notify your class
  - Write a statements that do something interesting
- This type of event handling (recommended in your book) uses **inner** classes for each button (Section 9.7)

Step 1: Create a Listener

- The listener class will be defined **inside** your class
  - Add **implements ActionListener** to class header
  - Add a method **actionPerformed()** inside the class
    - **Must** have one **ActionEvent** parameter
  - Add **import java.awt.event.*** to the list of imports

```
class MyHandler implements ActionListener
{
   public void actionPerformed(ActionEvent e)
   {
   }
}
```

Express Yourself

- **Exercise 2**: create a button-handler class
  - Name the class **NewHand**
  - Place it right before the **main()** method
  - Compile or run your program
  - Switch to **Resource View** in Eclipse
  - Locate the **NewHand** class
  - Shoot a picture of the resource window
- Notice that inner-classes are **automatically** named
  - **OuterClass$InnerClass.class**

Step 2 : Subscribe

- The **JButton** needs to know that you want events
  - This works like subscribing to the paper
  - You may be able to read it, but unless you "sign up" it won't be delivered to your door
- Add code like this to **hookup** the button:

```
ActionListener handler = new MyHandler();
theBtn.addActionListener(handler);
```

  - Creates an instance of the **MyHandler** class and tells the **theBtn** to notify it whenever it is clicked

Express Yourself

- **Exercise 3**: hook up **newHandBtn**.
  - In the **init()** method create an instance of your **NewHand** class
  - Make the variable type **ActionListener**
  - Name the variable **newHandListener**
  - Use the **NewHand** constructor
  - Call **newHandBtn**'s **addActionListener** method
  - Pass your **newHandListener** as the parameter
  - Shoot a screen-shot of your code

Step 3 : Respond

- How does your button "notify" you when it's clicked?
- By calling a method with this **signature**:

```
public void actionPerformed(ActionEvent ae)
{
    // your code goes here
}
```

- The inner class has access to outer instance variables
  - Means we can access our deck and table

Express Yourself

- **Exercise 4**: add code inside **actionPerformed()** to respond to clicking the **newHandBtn**
  - If **deck** is **null** or it has fewer than 2 cards, call the **newDeck()** method
  - Create a new **BlackjackHand** object and assign it to the **player** variable
  - Deal two cards from the deck and add them to the player's hand
  - Call **showCards** with the player and the **playerPile**
  - Call the **showScore** method and pass **player**
  - Run, click **newDeck** and shoot a screenshot

JButton Event Enabling

- Often, you'll want to **enable** or **disable** buttons depending on the results of a particular action
- Do that with the **JButton**'s **setEnabled()** method

```
newHandBtn.setEnabled(true); // active
hitBtn.setEnabled(false);  // deactivate
```

- **Exercise 5**: modify **showScore()** method
  - Score 21: Win message, hit and stand off, new on
  - Score < 21: Hit or stand activated, new off
  - Score > 21: Bust message, hit & stand off

Anonymous Objects

- Why do we "name" things in our programs?
  - To make the intent clearer, or to reuse the item
  - When you don't need reuse, avoid creating a variable
- Don't use **newHandListener** variable after button is hooked up, so you can simply construct an **anonymous NewHand** object when you call **addActionListener()** like this:

```
newHandBtn.addActionListener(
                        new NewHand());
```

Anonymous Classes

- Since we don't create multiple **NewHand** objects, you can remove the class name as well like this:
  - Note that this works for interfaces and inheritance

```
btn.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent ae)
    {
        // Carry out your actions here
    }
});
```

Express Yourself

- **Exercise 6**: let's hook up an anonymous event handler for the **hitBtn**. Here's what you should do
  - In **init()** method, call **hitBtn.addActionListener()**
  - Inside the parameter list, create an **ActionListener()** anonymous class with **actionPerformed()**
  - If the deck has less than 1 card, call **newDeck**
  - Add the top card to the player's hand
  - Pass **player** and **playerPile** to **showCards**, and the **player** to **showScore**
  - Run and shoot a screen-shot. Make sure you can hit.

Handling the Stand Button

- **Exercise 7**: hook up the **standBtn** to finish the game
  - Call **newDeck** if there are fewer than 2 cards
  - Create a new hand for the dealer and add two cards
  - Use a loop to keep adding cards to the dealer's hand while the score is <= 17. Make sure you call **newDeck** if you run out of cards
  - Display the dealers cards in the **dealerPile**
  - Call the **scoreGame()** method, passing the player and the dealer to the method
  - Write a skeleton for **scoreGame()**, run and shoot a screen-shot

Scoring the Game

- **Exercise 8**: write the code for **scoreGame()**:
  - Get the points for both hands and store in variables
  - If dealer is > 21, then dealer busts
  - If dealer and player have same points, then a tie
  - If player is < dealer, then player loses
  - If dealer is < player then player wins
  - Display appropriate messages
  - Disable the Hit and Stand buttons
  - Enable the New Hand button
  - Run and shoot a screen-shot

Event Adapter Classes

- Most event-handling interfaces require you to implement **more** than one method:
  - **MouseMotionListener** has **two** (RedDot)
  - **KeyListener** has **three**
  - **MouseListener** has **five**
  - **WindowListener** has **seven**
- Often, you'll just write dummy methods for all but one
  - Remember, you can't skip any of the methods!
- An **event adapter** is a class that does this for you

Using an Event Adapter

- Java provides adapters for all event-handling interfaces that require more than a single method
  - **ComponentAdapter**, **ContainerAdapter**, **FocusAdapter**, **KeyAdapter**, **MouseAdapter**, **MouseMotionAdapter**, and **WindowAdapter**
- To use an event adapter you must:
  - Create an inner class that extends one of the adapters
  - Override the methods of interest in your definition
  - Create an object of your inner class and hook it up
- **Exercise 9**: modify **Dot** to use an adaptor

Exception Handling

Dealing with Runtime Errors

Runtime Errors

- Runtime errors occur when your program runs
  - Cannot be caught by the Java compiler
- Three categories of runtime errors
  - Errors that cause the JVM to print an error message
    - Called runtime **exceptions**; your program "crashes"
  - Errors that cause your program to hang
  - Errors that cause your program to behave incorrectly
    - These are known as **logic** errors

Runtime Exceptions

- JVM notices a problem and notifies you
  - Provides opportunity intercept and correct
- **Exercise 10**: Open and run **RuntimeExceptions.java**
  - Press the "Nobody's Home" button
  - JVM Message: **NullPointerException**
  - Snap a picture and paste it in your document
- What is a **NullPointerException**?
  - Simply means an "uninitialized object"
- Solution: locate line, find object, fix your code

Other Runtime Exceptions

- **Exercise 11**: Press the "Problems with Math" button
  - JVM reports an **ArithmeticException**
  - Caused by **divide by zero** ; solution? Don't do that!!!
  - Snap and paste a picture, then fix and paste the code
- **Exercise 12**: Press the "How Much is a Cat" button
  - Paste a picture of the exception
  - Different than the other two exceptions
  - Not designed to protect integrity of JVM or universe
  - **NumberFormatException**: "**I don't know how**"
  - Solution? Use try-catch to recover (soon…)

Hanging Programs

- Programs that stop responding
  - Obvious that something is wrong
  - No helpful message from the JVM
- Two general reasons for hanging programs
  - **Most common**: you have written an endless loop
  - You have a "system" error
- System errors: cause the O/S to stop responding
  - Depleting an O/S resource (files, database, graphics)
  - Causing a deadlock (files, databases, threads)

Logical Errors

- Logical errors are the most difficult to fix
  - Cause your program to run incorrectly
  - No error messages are printed
  - No obvious malfunction such as hanging
  - Incorrect results may be produced only sporadically
- Solution for logical problems?
  - Call it a "feature" (ancient computer joke)
  - Rigorous testing to find logical errors
  - Learn to use JUnit religiously (TDD)
  - Debugging strategy to remove them

Exceptional Situations

- Sometimes, runtime errors are caused by **unusual situations**, not programming errors
- For example: writing data to a file
  - Most of the time things go uneventfully, but...
    - The disk may be full
    - There may be a hardware error
    - The file may have been changed to read-only
- **Fragile** code ignores the possibility of problems
- **Robust** code anticipates such problems

Completion Codes

- Tradition method is to use "completion codes"

```
GET A FILENAME
OPEN THE FILE
IF THERE IS NO ERROR OPENING THE FILE
        READ SOME DATA
        IF THERE IS NO ERROR READING THE DATA
                PROCESS THE DATA
                WRITE THE DATA
                IF THERE IS NO ERROR WRITING THE DATA
                        CLOSE THE FILE
                        IF THERE IS NO ERROR CLOSING FILE
                                RETURN
```

Examining Completion Codes

- Things to notice with completion codes
  - Almost every step **may fail**, except "PROCESS DATA"
    - Program just be prepared to deal with failure
  - Very difficult to determine "normal" actions
    - Program so concerned with things that **can** go wrong that it is hard to tell if you are doing the right things in the right order
  - Difficult to use if library function contains such code
    - Function must return several error codes so that user can tell what went wrong

The Exceptional Method

- Using exceptions the same code looks like this:

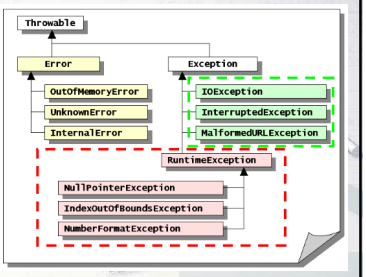
```
TRY TO DO THESE THINGS:
   GET A FILENAME
   OPEN THE FILE
   READ SOME DATA
   PROCESS THE DATA
   WRITE THE DATA
   CLOSE THE FILE
RETURN
IF ERROR OPENING THE FILE THEN ...
IF ERROR READING THE DATA THEN ...
IF ERROR WRITING THE DATA THEN ...
IF ERROR CLOSING THE FILE THEN ...
```

Examining Exceptions

- Compare the two methods of handling "bad things"
- Code that uses the exception strategy:
  - Is shorter and easier to read
  - Makes the normal logic the **main focus** of the method. In the completion code version, the error-handling code **obscures** the normal actions
  - Allows you to decide whether to handle the problem or defer handling the error on a case-by-case basis
    - Ideal for library-based code

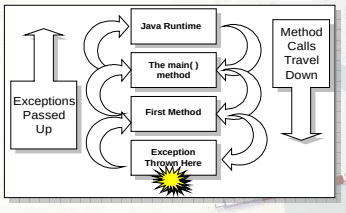
Throwable Objects

- In Java, when a runtime error occurs, the JVM notices and creates one of two types of **Throwable** objects
  - An **Error** object or an **Exception** object



Throwing Exceptions

- The JVM then **throws** the exception or error object back **up** the call stack until it is "caught"
  - If it is not caught then the JVM runtime system prints an error message



The Error Classes

- Notice the different kinds of **Error** classes
  - **InternalError**
  - **OutOfMemoryError**
  - **UnknownError**
- Fatal situations that you cannot normally deal with
  - How do you fix an "Unknown" error, for instance?
- Normally, ignore the **Error** classes
- Simply allow to percolate up to runtime

The Exception Classes

- Situations you **may** want to handle in your code
- Two general categories
  - **Unchecked** exceptions (**RuntimeException**)
    - You are **not required** to specifically deal with these
    - Most of the time you, treat them like **Error** object
    - **NumberFormatException** is one you probably will want to handle in your own code
  - **Checked** exceptions
    - Exceptions that **will** commonly occur
    - You **must** explicitly handle all checked exceptions

Using try-catch

- In Java, you **handle** exceptions with **try-catch**

```
try
{
    riskyBusiness( );
}
catch (SomeException e)
{
    // Handle problems here
}
```

Statement or methods that may throw exceptions

Exception argument to be caught

Code to deal with the problem

A try Block Example

- Methods throwing checked exceptions (**IOException** and **InterruptedException**) **must** be in a **try**
  - Unchecked exceptions **may** be placed in a try

```
int a = 3, b = 0, n = 0, x = 0;
try
{
  n = System.in.read(); // IOException
  Thread.sleep(100);    // InterruptedException
  x = a / b;            // ArithmeticException
}
```

A catch Block Example

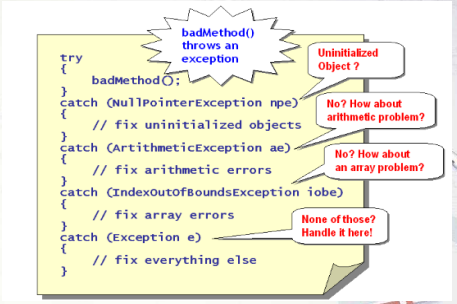
- You **must** provide a catch block for **every** checked exception that **may be thrown** inside your **try** block.
  - Could also catch the **Exception** superclass

```
int a = 3, b = 0, n = 0, x = 0;
try
{
  n = System.in.read(); // IOException
  Thread.sleep(100);    // InterruptedException
  x = a / b;            // ArithmeticException
}
catch (IOException ioe) {/*System.in.read failure */}
catch (InterruptedException ie) {/*sleep failure */}
```

Express Yourself

- **Exercise 13**: open the console app named **GetNums**, and use **try-catch** to complete these requirements:
  - Ask the user to input an integer
  - Read the user's input into a **String** variable
  - Convert input to an **int**, using **Integer.parseInt()**
  - If the input is invalid ("like cat") then print an error message and ask the user to input the number again
  - Continue asking until a valid number is entered
  - Run, snap a pic, then paste your code.

Catching Multiple Exceptions



The throw Statement

- Allows you to "trigger" exceptions in your code

```
public Widget makeWidget(int widgetType)
{
    // only valid types are 1...4
    if (widgetType < 1 || widgetType > 4)
        throw new IllegalArgumentException();

    // Remaining code skipped when thrown
}
```

- Your **catch** code can call exception object's methods, such as **getMessage()**, and **getCause()**

Express Yourself

- **Exercise 14**: Add validation to **GetNums**
  - Use the **throws** statement to make sure that only numbers between 1 and 10 (inclusive) can be entered. Add a separate catch block to handle an "out of bounds" error and use **getMessage()** to print the error message for the user.
  - Run and snap a pic, then paste your code.

The throws Clause

- When designing a method, you can handle exceptions using **try-catch**, or you can **propagate** the exception
- Add **throws Exception** to your method header
  - Allows you to skip using **try-catch** in your method
  - Required if you throw any checked exception
- Means that caller must use **try-catch** or **throw** the exception as well, but your code doesn't have to
  - If caller doesn't handle, will eventually be handled by the JVM, just like errors and unchecked exceptions
- See illustration on next slide

Using the throws Clause

Thrown exception is returned to the caller

```
public int readInt( )  throws IOException
{
    int result = 0;
    String s = "";
    int ch = System.in.read( );

    while ( ch != '\n' && ch != '\r' )
    {
        s += (char) ch;
        ch = System.in.read( );
    }

    result = Integer.parseInt(s);
    return result;
}
```

May throw an IOException

May throw an IOException

May throw a NumberFormatException

Now, Finally, finally

- When an exception is thrown
  - Control jumps to the **catch** block that handles it
  - Code that manages **resources** may be skipped

```
openFile();
readData();  // Exception thrown here
closeFile();  // This is skipped. File still open
```

- The **try** has an optional **finally** part
  - This will always be called
  - Use to handle resource depletion

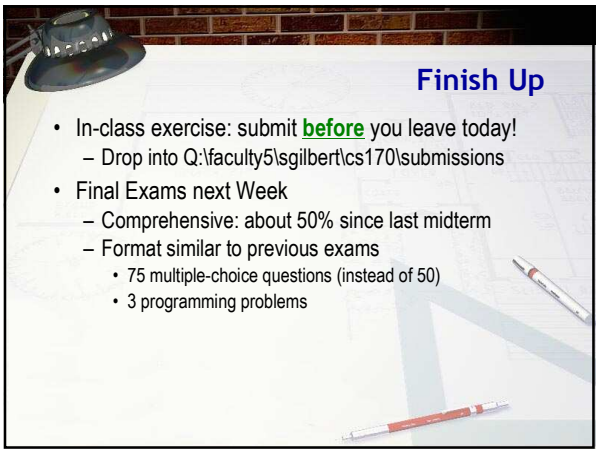
Using finally

A "resource" is allocated in the try block

```
try
{
    Resource r = new Resource();
    doSomethingWith( r );
}
catch (SomeException e)
{
    // Cope with tragic failure
}
finally
{
    r.dispose();
}
```

Error may happen here

Resource ALWAYS released here


Finish Up
• In-class exercise: submit before you leave today!
– Drop into Q:\faculty5\sgilbert\cs170\submissions
• Final Exams next Week
– Comprehensive: about 50% since last midterm
– Format similar to previous exams
• 75 multiple-choice questions (instead of 50)
• 3 programming problems